ServiceNow GlideUser API

Ultimate Scratch-to-Advanced Reference Guide

This reference manual covers both client-side and server-side GlideUser APIs in detail, explaining variables, properties, methods, checks, and real-world code use cases.

1. GlideUser Architecture & Scope Boundaries

ServiceNow splits JavaScript execution between client-side scripts (runs in browser) and server-side scripts (runs directly on the cloud server database).

Feature	Client-Side (g_user)	Server-Side (gs.getUser())
Where it runs	User's browser layout window session.	ServiceNow private cloud instance nodes.
Speed Context	Instant access via cache; does not impact load.	Queries database; requires CPU/RAM resources.
Trust Factor	Untrusted: users can bypass via F12 developer tool console.	Fully trusted: users cannot access server memory.
Main Use Case	Dynamic UI changes, field visibility, forms formatting.	Database security (ACLs), business calculations, validation.

2. Client-Side GlideUser (g_user)

The client-side g_user object contains properties and methods describing the active user session.

A. Properties

- g_user.firstName: First name string of the current user record.
- g_user.lastName: Last name string.
- g_user.userID: Sys ID (sys_id) of the user record.
- g_user.userName: Login username (e.g. 'admin').

B. Methods

- g_user.getFullName(): Combines first name and last name into a single display string.
- g_user.hasRole(roleName): Returns true if the user holds the role. Admins automatically evaluate to true.
- g_user.hasRoleExactly(roleName): Returns true only if the user holds the role explicitly (does not bypass for Admins).
- g_user.hasRoles(): Returns true if the user is assigned any roles (excluding 'public').
- g_user.hasRoleFromList(rolesList): Accepts a comma-separated list of roles and returns true if the user has any of them.

```
// Reading properties and calling methods
var fName = g_user.firstName;
var sysId = g_user.userID;
var isAgent = g_user.hasRole('itil');
var isExplicitSec = g_user.hasRoleExactly('security_admin');
```

C. Client Use Cases: Easy to Hard

Use Case 1: Display Greeting Banner [Easy]

Goal: Display an info greeting banner on the Incident form when it loads, addressing the user by their full name.

```
function onLoad() {
    var fullName = g_user.getFullName();
    g_form.addInfoMessage('Welcome back, ' + fullName + '! Review your open tickets.');
}
```

Use Case 2: Field Controls Based on Roles [Medium]

Goal: When the Incident form loads, check if the user is a licensed ITIL agent. If not, hide the internal 'Work Notes' field and make 'Additional Comments' mandatory.

```
function onLoad() {
    if (g_user.hasRole('itil')) {
        g_form.setDisplay('work_notes', true);
    } else {
        g_form.setDisplay('work_notes', false);
        g_form.setMandatory('comments', true);
    }
}
```

Use Case 3: Admin Lockout & Explicit Roles [Hard]

Goal: A custom field 'u_secure_override' is only editable by users who explicitly hold the 'security_admin' role. Standard Administrators should be locked out unless they hold this role explicitly.

```
function onLoad() {
    // hasRoleExactly prevents admin bypass
    var hasAccess = g_user.hasRoleExactly('security_admin');

    if (hasAccess) {
        g_form.setReadOnly('u_secure_override', false);
        g_form.showFieldMsg('u_secure_override', 'Secure access session active', 'info');
    } else {
        g_form.setReadOnly('u_secure_override', true);
    }
}
```

3. Server-Side GlideUser (gs.getUser())

Retrieve the user's database object using GlideSystem on the server side.

A. Profile Retrieval Methods

- gs.getUserID(): Returns the Sys ID of the current user.
- gs.getUserName(): Returns the login username.
- gs.getUser().getDisplayName(): Display Name.
- gs.getUser().getEmail(): User email address.
- gs.getUser().getDepartmentID(): Department sys_id.
- gs.getUser().getCompanyID(): Company sys_id.
- gs.getUser().getManagerID(): Manager user sys_id.

```
var user = gs.getUser();
gs.info('Email: ' + user.getEmail());
gs.info('Department ID: ' + user.getDepartmentID());
```

B. Group & Role Checking Methods

- gs.getUser().isMemberOf(group): Returns true if the user is a member of the group (accepts Group Name or Group Sys ID).
- gs.getUser().getMyGroups(): Returns a Java ArrayList of the Sys IDs of all groups the user belongs to.
- gs.getUser().getRoles(): Returns a Java ArrayList containing all roles assigned to the user.

C. Server Use Cases: Easy to Hard

Use Case 4: Populate Department & Company [Easy]

Goal: When creating a record on a custom table, auto-populate Company and Department based on the logged-in user's profile.

```
// Business Rule (Before Insert)
(function executeRule(current, previous) {
    var userObj = gs.getUser();
    current.setValue('u_department', userObj.getDepartmentID());
    current.setValue('u_company', userObj.getCompanyID());
})(current, previous);
```

Use Case 5: Restrict Queue to Group Members [Medium]

Goal: Prevent users from saving a ticket assigned to the 'DB Analysis' group unless the logged-in user is a member of that group.

```
// Business Rule (Before Insert/Update)
(function executeRule(current, previous) {
    var groupName = current.getDisplayValue('assignment_group');

    if (groupName === 'DB Analysis') {
        var isMember = gs.getUser().isMemberOf('DB Analysis');
        if (!isMember) {
            gs.addErrorMessage('Only members of DB Analysis can assign to this queue.');
            current.setAbortAction(true); // Blocks the database save transaction
        }
    }
})(current, previous);
```

Use Case 6: Custom ACL Record Ownership Filter [Hard]

Goal: A user can write to records on the Custom Asset table only if: 1. They are a member of the record's Ownership Group, OR 2. They are the manager of the record's Assignee.

```
// ACL Script (Server-Side Write Rule)
answer = false; // Default blocked

var ownershipGroup = current.getValue('u_ownership_group');
var assigneeManager = '';

// Query assignee manager reference
var assigneeGR = new GlideRecord('sys_user');
if (assigneeGR.get(current.getValue('assigned_to'))) {
    assigneeManager = assigneeGR.getValue('manager');
}

// 1. Check if user is a member of ownership group
if (ownershipGroup && gs.getUser().isMemberOf(ownershipGroup)) {
    answer = true;
}

// 2. Check if user is the direct manager of the assignee
if (assigneeManager && gs.getUserID() === assigneeManager) {
    answer = true;
}
```

SECURITY WARNING: Always implement security verification on the server. Client scripts (g_user) are for user interaction, whereas server scripts (gs.getUser()) enforce data integrity and protection.